

NAME: OYEKUNLE OLUWASEUN JOSEPH.

PURPOSE: ASSIGNMENT.

DATE: 18 OF AUGUST, 2026.

DEPARTMENT: ICT/MIS

SUPERVISOR: MR BAKARE OLUWATOBI

MATRIC NUMBER: AUL/CMP/23/092

JavaScript Fundamentals: Variables, Coercion, Operators, and Conditional Logic

Introduction

JavaScript is a dynamically typed, loosely typed language, and much of what makes it both flexible and occasionally confusing traces back to those two properties. This essay walks through four foundational areas of the language — how variables are declared and scoped, how JavaScript quietly converts between types when it evaluates expressions, how the core arithmetic and assignment operators behave, and how the language makes decisions through conditional statements. Each section builds toward the same goal: writing JavaScript that behaves the way you expect it to, rather than the way it happens to.

1. Declaring Variables: `var`, `let`, and `const`

Before ES6 (2015), JavaScript had only one way to declare a variable: `var`. A variable declared with `var` is scoped to the nearest **function**, not to the nearest block of code. This means a `var` declared inside an `if` statement or a `for` loop is still visible outside that block, as long as it is within the same function:

```
function testVar() {  
  if (true) {  
    var x = 10;  
  }  
  console.log(x); // 10 – accessible outside the if-block  
}  
testVar();
```

`var` is also **hoisted**: the JavaScript engine moves the declaration (though not the assignment) to the top of the function before execution, and initializes it as `undefined`. This is why referencing a `var` before its declaration line does not throw an error, it simply returns `undefined` — a behaviour that can mask bugs rather than surface them.

ES6 introduced `let` and `const` specifically to fix this. Both are **block-scoped**, meaning they only exist within the nearest pair of curly braces `{ }` — whether that block belongs to an `if`, a `for` loop, or a standalone block. Referencing a `let` outside the block in which it was declared throws a `ReferenceError`, which is exactly the safety net `var` was missing:

```
function testLet() {  
  if (true) {  
    let y = 10;  
  }  
}
```

```
console.log(y); // ReferenceError: y is not defined
}
```

let and **const** are also hoisted, but unlike **var** they are not initialized until their declaration line is actually executed. The gap between the start of the block and that line is known as the **Temporal Dead Zone**, and trying to access the variable during that gap throws an error rather than silently returning **undefined**.

The difference between **let** and **const** is simpler: **let** allows reassignment, **const** does not. Once a **const** variable is initialized, attempting to assign it a new value throws an error:

```
const PI = 3.14159;
PI = 3.14; // TypeError: Assignment to constant variable.
```

It is worth being precise about what **const** actually locks down. It freezes the **binding** — the name-to-value link — not the contents of the value itself. If that value is an object or array, its properties or elements can still be changed freely, because doing so does not reassign the variable:

```
const person = { name: "Ada" };
person.name = "Grace"; // Allowed – this mutates a property, not the binding
console.log(person);    // { name: "Grace" }

person = {};             // TypeError – this would reassign the binding itself
```

In modern JavaScript, the common convention is to default to **const**, switch to **let** only when a variable genuinely needs to be reassigned (such as a loop counter or an accumulator), and avoid **var** altogether. This convention exists precisely because block scoping and immutable bindings catch mistakes earlier — often at the moment the code is written, rather than somewhere deep in a debugging session.

2. Type Coercion in Practice: "HELLO" + 5

JavaScript almost never refuses to evaluate an expression just because the operand types don't match. Instead, it tries to make sense of the mismatch through a process called **type coercion** — silently converting one or both operands so the operator can proceed. The behaviour of **"HELLO" + 5** is the clearest illustration of this, because the **+** operator is deliberately overloaded to serve two different purposes: numeric addition, and string concatenation.

When at least one operand of **+** is a string, JavaScript chooses concatenation. It converts the other operand to a string and joins the two:

```
console.log("HELLO" + 5);           // "HELLO5"
console.log(typeof ("HELLO" + 5));  // "string"
```

No error is thrown, because this is not a type mismatch as far as `+` is concerned — it is simply the operator doing exactly what it is designed to do with a string operand.

If the intent is to force a *numeric* interpretation instead, switching to an operator that has no string meaning — `-`, `*`, `/`, `%`, or `**` — changes the outcome. These operators only make sense for numbers, so JavaScript attempts to convert the string operand to a number rather than concatenate:

```
console.log("HELLO" - 5);           // NaN
console.log(typeof ("HELLO" - 5)); // "number"
```

The result is `NaN` ("Not-a-Number"), because `"HELLO"` has no valid numeric equivalent. Critically, this still does not throw an error. `NaN` is itself a value of type `"number"` — JavaScript's way of saying "this is a numeric slot, but the value inside it is invalid," rather than refusing to produce a result at all. This is a meaningful contrast with statically typed languages like Java, where converting a non-numeric string to a number raises an exception.

The same idea applies if the conversion is done explicitly and *before* the arithmetic, rather than letting the operator trigger it implicitly. `Number()`, `parseInt()`, and the unary `+` operator all attempt to convert a string to a number on demand:

```
let converted = Number("HELLO"); // convert the string to a number first
console.log(converted);           // NaN

let result = converted + 5;       // then perform the arithmetic
console.log(result);             // NaN
console.log(typeof result);      // "number"
```

`parseInt("HELLO")` and `+"HELLO"` behave identically, returning `NaN` for the same reason: there is no digit sequence anywhere in `"HELLO"` to extract. Explicit conversion does not change what is mathematically possible — it only changes *when* the conversion happens and makes the intent of the code clearer to a reader.

Where explicit conversion genuinely pays off is with strings that *do* represent numbers. Converting `"5"` produces a real, usable number rather than `NaN`:

```
let convertedGood = Number("5"); // 5 – a valid number
console.log(convertedGood + 5);   // 10 – a real, usable digit output
```

Placed side by side, the two cases make the underlying rule clear: `Number()` (and its relatives) only produce a *meaningful* digit output when the source string actually encodes a number. `"HELLO"` doesn't

encode one, so no amount of conversion will manufacture one — the honest result is **NaN**, delivered without an error, which is precisely how JavaScript is designed to fail: quietly, with a special value, rather than by halting execution.

3. Operators in Action: ******, **%**, *****, **+=**, **=**, **/=**, **-=**

JavaScript's operators fall broadly into two families relevant here: arithmetic operators, which compute a new value, and assignment operators, which store a value into a variable — including compound forms that compute and store in a single step.

Among the arithmetic operators, ****** is the exponentiation operator, raising its left operand to the power of its right operand:

```
let power = 5 ** 2;
console.log(power); // 25 – 5 raised to the power of 2
```

%, the modulus operator, returns the *remainder* left over after dividing the left operand by the right — useful for tasks like checking whether a number is even, or wrapping a value around a fixed range:

```
let remainder = 10 % 3;
console.log(remainder); // 1 – 10 divided by 3 leaves a remainder of 1
```

*****, ordinary multiplication, needs little introduction:

```
let product = 4 * 3;
console.log(product); // 12
```

The assignment operators build on top of these. The plain **=** operator simply stores a value in a variable — it is the most basic operator in the language, and every other assignment form is a shorthand built around it:

```
let username;
username = "chinedu_dev";
console.log(username); // "chinedu_dev"
```

+=, **-=**, and **/=** are **compound assignment operators**: each one performs an arithmetic operation and reassigns the result back to the same variable, in one expression instead of two. **+=** adds and reassigns:

```
let score = 5;
score += 3;           // equivalent to: score = score + 3
console.log(score); // 8
```

`-=` subtracts and reassigns:

```
let balance = 100;
balance -= 40;      // equivalent to: balance = balance - 40
console.log(balance); // 60
```

and `/=` divides and reassigns:

```
let total = 20;
total /= 5;      // equivalent to: total = total / 5
console.log(total); // 4
```

The value of compound operators is mostly readability and intent: `score += 3` communicates "increase score by 3" more directly than `score = score + 3`, even though the two are functionally identical.

4. Conditional Statements: `switch`, Ternary, and Boolean Logic

Every non-trivial program needs to make decisions, and JavaScript offers several ways to express them depending on how many outcomes are possible and how much code each outcome requires.

`switch`

A `switch` statement is best suited to checking **one variable against several fixed values** — it reads more clearly than a long chain of `if...else if` statements once there are more than two or three branches. Execution jumps to the matching `case`, and `default` catches anything that doesn't match:

```
function getDayType(day) {
  switch (day) {
    case "Saturday":
    case "Sunday":
      return "Weekend";
    default:
      return "Weekday";
  }
}

console.log(getDayType("Sunday")); // "Weekend"
console.log(getDayType("Monday")); // "Weekday"
```

Stacking `case "Saturday":` directly above `case "Sunday":` with no `return` in between lets both values share the same result — a pattern often called "case fall-through," used deliberately here rather than by accident.

Ternary Operator

The ternary operator is a compact, single-expression alternative to a short `if...else`. Its syntax is `condition ? valueIfTrue : valueIfFalse`, and it is most readable when both branches are simple values rather than multi-step logic:

```
let age = 20;
let access = age >= 18 ? "Granted" : "Denied";
console.log(access); // "Granted"
```

Because it is an *expression* rather than a *statement*, a ternary can be embedded directly inside a template string, a function argument, or a variable assignment — something a full `if...else` block cannot do without extra scaffolding. This convenience comes with a caution, though: nesting multiple ternaries inside one another quickly becomes hard to read, and a regular `if...else` is usually the better choice once the logic grows beyond a single condition.

Boolean (Logical) Conditionals

The most fundamental form of decision-making in JavaScript is the plain `if...else` statement, which branches based on whether an expression evaluates to `true`, `false`, or a value the language treats as one of the two ("truthy" or "falsy"):

```
let isLoggedIn = true;

if (isLoggedIn) {
  console.log("Welcome back!");
} else {
  console.log("Please log in.");
}
```

JavaScript doesn't require the condition to be a strict boolean — values like `0`, `""`, `null`, `undefined`, and `NaN` are all treated as falsy, while virtually everything else, including non-empty strings and non-zero numbers, is treated as truthy. This is the same coercion instinct discussed in Question 2, applied to control flow instead of arithmetic: JavaScript would rather make a reasonable guess about what a value "means" than refuse to proceed.

Conclusion

A single thread runs through all four topics covered here: JavaScript consistently favors flexibility and graceful degradation over strict enforcement. `let` and `const` were introduced to add discipline back into variable scoping without abandoning the language's dynamic nature. The behaviour of `"HELLO" + 5` and its numeric variants shows a language that coerces types rather than rejecting mismatched operands

outright. The arithmetic and assignment operators offer both explicit and shorthand ways to compute and store values. And the three conditional forms — `switch`, ternary, and boolean `if...else` — each trade off verbosity against clarity depending on how many branches a decision needs. Understanding *why* JavaScript behaves this way, rather than only *what* it does, is what turns these four topics from separate facts into one coherent picture of the language.